UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

BugBoard26

Documentazione del progetto di Ingegneria del Software

Mario Lombardo
Matricola N. N86004029

Giuseppe Pollio
Matricola N. N86004953

Antonio Paudice
Matricola N. N86004566

Professori:

Porfiro Tramontana
Breve Bernardo

Università degli Studi di Napoli Federico II
Classe di Laurea in Scienze e Tecnologie Informatiche L-31

Anno Accademico 2025/2026

Pagina vuota

Indice

Lista delle Figure **iv**

Lista delle Tabelle **v**

Glossario **vi**

1 Introduzione **1**

2 Descrizione Requisiti **2**

2.1 Requisiti di sistema 2

2.2 Requisiti Non-Funzionali 3

3 Utenti del sistema **4**

4 Casi d'uso **6**

4.1 Use-Case diagram completo 6

4.2 Tabella completa Use-Case 7

5 Prototipazione via Mockup **9**

5.1 Descrizione dettagliata dei Mockup 9

5.1.1 Schermata di Login 9

5.1.2 Area progetti 10

5.1.3 Pagina issues del Progetto 10

5.1.4 Modulo creazione Issue 12

5.1.5 Aggiornamento lista Issue 12

5.1.6 Errore durante la compilazione del modulo (creazione issue) 12

5.1.7 Errore interno, per qualche motivo i dati non vengono salvati nel database . . . 12

6 Design Del Sistema **14**

6.1 Descrizione dell'architettura 14

6.1.1 Deployment diagram 15

6.2 Criteri di design adottati 16

6.3 Scelte tecnonlogiche 16

6.3.1 Frontend 16

6.3.2 Backend . . . 16
6.3.3 Deployment e Sviluppo . . . 17
6.4 Comunicazione fra livelli . . . 18
6.4.1 Sicurezza e Controllo degli Accessi . . . 18

7 Design del Software **19**
7.1 Scema del Database e Modello di Dominio . . . 19
7.1.1 Class Diagram del database . . . 19
7.1.2 Descrizione entità . . . 19
7.1.3 Descrizione Relazioni . . . 20
7.2 Organizzazione Architetturale dei Package . . . 21
7.2.1 Package Diagram . . . 21
7.2.2 Backend (it.unina.bugboard26) . . . 21
7.2.3 Frontend (web) . . . 22
7.2.4 Il flusso delle Dipendenze . . . 22
7.2.5 Design Pattern applicati . . . 22
7.3 Sequence Diagrams . . . 23
7.3.1 Sequence Diagram: User Login . . . 23
7.3.2 Sequence Diagram: Creazione Segnalazione Bug/Issue . . . 25

8 Infrastruttura, DevOps e Continuous Integration **27**
8.1 Contrainerizzazione tramite Docker . . . 27
8.2 Automazione del Ciclo di Vita e Makefile . . . 27
8.3 Pipeline di Continuous Integration (CI) . . . 27

9 Piano di collaudo **29**
9.1 Metodologia e Perimetro di Collaudo . . . 29
9.2 Strategie di Isolamento e Classi di Equivalenza (Black Box) . . . 29
9.3 Metodi Sottoposti a Test e Casi Pianificati . . . 29
9.3.1 Servizio di Gestione Segnalazioni (`IssueService.update()`) . . . 30
9.3.2 Servizio di Autorizzazione: PermissionService.canModifyIssue() . . . 31
9.3.3 Servizio di Esportazione: ExportService.exportCsv() . . . 32

10 Analisi statistica e Qualità del Codice (SonarQube) **33**
10.0.1 Security and Vulnaribilities (Sicurezza) . . . 33
10.0.2 Reliability and Bugs (Affidabilità) . . . 33
10.0.3 Manintainability and Code Smells (Manutenibilità e Complessità) . . . 33
10.0.4 Test Coverage (Copertura del Codice) . . . 34

11 Gestione della Configurazione e Versioning **35**

11.0.1 Repository Ufficiale 35

11.0.2 Metriche, Statistiche, Workflow 35

Lista delle Figure

4.1 Use-Case Completo 6

5.1 Schermata di login 9
5.2 Area progetti 10
5.3 Pagina Issue 11
5.4 Creazione Issue 11
5.5 Aggiornamento lista 12
5.6 Errore 1 13
5.7 Errore 2 13

6.1 Deployment Diagram 15

7.1 Class Diagram del database 19
7.2 Package diagram 21
7.3 Sequence diagram Autenticazione Utente 23
7.4 Sequence Diagram Creazione Segnalazione Bug/Issue 25

Lista delle Tabelle

4.1 Tabella completa Use-Case 7
4.2 Cockburn template Use-Case N.02, Segnalazione di una nuova issue 8

Glossario

API

Application Programming Interface - Interfaccia di programmazione delle applicazioni

UML

Unified Modeling Language - Linguaggio di modellazione unificato utilizzato per la specifica, visualizzazione e documentazione dei sistemi software

Framework

Struttura di supporto su cui un software può essere organizzato e progettato

Repository

Archivio centralizzato dove vengono memorizzati e gestiti i dati del progetto

UI

User Interface - Interfaccia comprensiva di elementi col quale l'Utente controlla e comunica con il sistema

Pagina vuota

1

Introduzione

Il progetto **BugBoard26** ha come obiettivo la realizzazione di una piattaforma per la **gestione collaborativa di issue** in progetti software. Il sistema consente a team di sviluppo di **segnalare, assegnare, monitorare e risolvere problematiche** legate al ciclo di vita di un progetto.

Il documento illustra l'analisi e la specifica dei requisiti funzionali e non funzionali, la modellazione dei casi d'uso e la prototipazione delle interfacce.

2

Descrizione Requisiti

I requisiti del sistema definiscono ciò che l'applicazione deve fare e come deve operare.
I **Requisiti Di Sistema** descrivono le funzionalità principali e le caratteristiche tecniche necessarie al corretto funzionamento del software, quali autenticazione e gestione dei progetti/issues.
I **Requisiti Non-Funzionali** specificano le qualità del sistema, concentrandosi su come deve operare. Include aspetti come sicurezza, usabilità e performance, garantendone efficienza, stabilità e affidabilità.

2.1 Requisiti di sistema

- **Gestione Autenticazione:** Il sistema deve consentire il login a utenti tramite email e password, distinguendoli tra utenti *Normali* e *Admin*.

- **Gestione Progetti:** Schermata iniziale dov'è possibile visualizzare e cercare tutti i progetti a cui l'utente lavora, con informazioni annesse come il numero di *issues* aperte per progetto e il numero di membri che ci partecipano.
  Nel caso dell'*admin*, dev'essere possibile anche gestire progetti già esistenti o crearne di nuovi.

- **Creazione Issues:** Gli *admin* devono poter creare nuove *issues*, specificando tipo, priorità e una breve descrizione, mentre gli utenti normali possono segnalare nuove *issues* agli admin; opzionalmente si possono anche inserire un'immagine e uno stato.

- **Assegnazione Issues:** Gli *admin* devono poter assegnare *issues* a membri del progetto, notificando gli utenti interessati.

- **Gestione stati:** Il sistema deve consentire il cambiamento degli stati delle issueus, sia dagli *Admin* sia dagli utenti a cui è stata assegnata un issue.

- **Filtraggio Issues:** L'utente deve poter filtrare le issue per nome, tipo (Bug, Feature, Question, Documentation), stato (Aperta, In Progress, Risolta, Chiusa), priorità (Bassa, Media, Alta, Critica) o in base al tempo di creazione.

- **Archiviazione Issues:** L'*admin* dev'essere in grado di archiviare *issues* risolte, in modo tale che non siano più visibili nella lista principale ma rimangano consultabili.

- **Commenti e aggiornamenti:** Ogni Issue deve poter includere una sezione commenti/aggiornamenti, visibile a tutti i membri del progetto .

- **Esportazione dati:** Il sistema dev'essere in grado di consentire l'esportazione dati in vari formati, quali CVS, PDF o Excel.
- **Modalità Read-Only:** Il sistema deve permettere la creazione di utenti in sola lettura per esterni al team di sviluppo, che consente di visualizzare *issues*, progetti e commenti senza però permetterne la modifica; La creazione e gestione di questi account è a carico solo degli *Admin* del sistema

2.2 Requisiti Non-Funzionali

- **Sicurezza:** Tutti i dati di utenti (progetti, issues) sono accessibili solo previa autenticazione, utilizzando email e password memorizzate in forma cifrata.
- **Affidabilità:** Il sistema deve garantire registrazione e conservazione di tutte le modifiche apportate alle *issues* senza perdere informazioni
- **Performance:** Il caricamento dei progetti o delle *issues* non deve richiedere tempo eccessivo, con database di dimensioni standard
- **Manutenibilità:** Il sistema dev'essere sviluppata in modo modulare per semplificare eventuali manutenzioni e aggiornamenti futuri
- **Portabilità:** Dev'essere garantita l'esecuzione del sistema su qualsiasi browser moderno
- **Backup e recupero dati:** Devono essere garantiti meccanismi di backup automatico e di esportazione manuale dei dati in vari formati per prevenire perdite accidentali

3

Utenti del sistema

Per il corretto sviluppo di **BugBoard26**, è necessario definire quali sono gli utenti che andranno a **utilizzare il sistema**, e di quali **competenze** o **esigenze** hanno bisogno

Il sistema è destinato a team di **sviluppo software** e, più in generale, a tutte le organizzazioni che necessitano di uno strumento collaborativo per la **gestione di *issues*, bug e richieste di miglioramento** nei propri progetti informatici.

All'interno del sistema sono stati individuati quattro *attori* principali, tutti con accesso a funzionalità diverse:

1. **Utente non autenticato:**
   - Ha accesso solo alla schermata di login
   - Non può accedere a dati a meno che non entra nel proprio account.
2. **Utente normale (*User*):**
   - Rappresenta il profilo standard utilizzato dalla maggior parte del team di sviluppo (sviluppatori, tester...)
   - Può visualizzare i progetti a cui partecipa e le *issues* a lui assegnate.
   - Può creare nuove *issues* secondo il procedimento già indicato.
   - Può filtrare e ordinare *issues* in base a vari criteri
   - Può aggiornare lo stato delle *issues* a lui assegnate
3. **Utente Admin:**
   - Dispone di privilegi avanzati per gestire al meglio l'intero sistema.
   - Ha come obiettivo di monitorare e coordinare il lavoro del team.
   - Ha completa gestione dei progetti, può crearli, eliminarli o modificarli.
   - Può creare account di qualsiasi tipo, specificandone email, password, e tipo di utenza (Normale/Admin/Read-Only).
   - Può assegnare *issues* a membri del team e contrassegnare eventuali duplicati.
4. **Utente in sola lettura (Read-Only):**

- Si tratta di un utente esterno al team di sviluppo, come uno stakeholder o un committente.
- Gli account di questa categoria possono essere gestiti e creati solo dagli *admin*
- Ha lo scopo di monitorare l'andamento del lavoro senza interferire con il lavoro del team
- Può visualizzare progetti, *issues* e commenti, senza però modificare o creare nulla

4

Casi d'uso

I **casi d'uso** descrivono **scenari concreti** di **interazione attori-sistema**, illustrando come il software risponde alle azioni dell'utente per soddisfare un obiettivo specifico.
Hanno il ruolo di far comprendere il **flusso operativo** dell'intero sistema.

4.1 Use-Case diagram completo

L'immagine comprende un **diagramma Use-Case** dell'intero sistema, comprendente di tutti gli *Attori* del sistema e degli 8 casi d'uso assegnati.

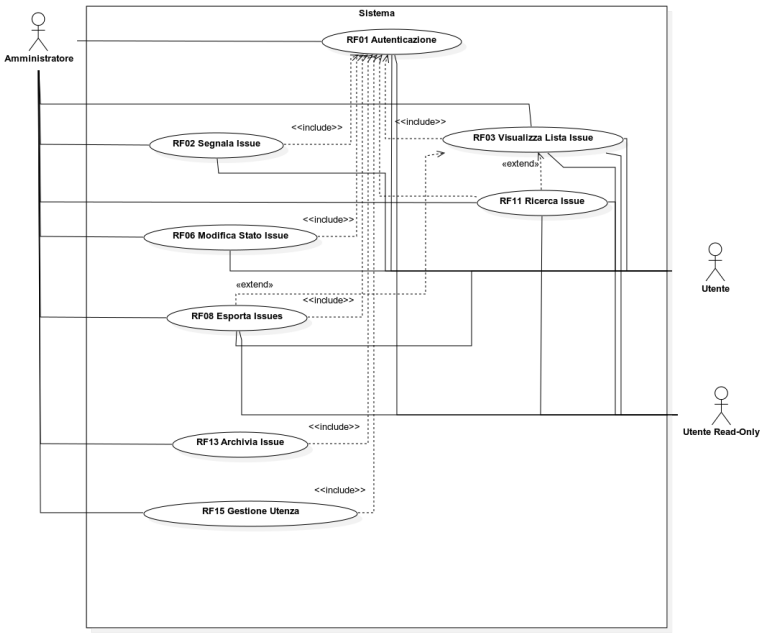

Figura 4.1: Use-Case Completo

4.2 Tabella completa Use-Case

La seguente tabella descrive nel dettaglio i casi d'uso presenti nell'immagine sopra

Tabella 4.1: Tabella completa Use-Case

ID	Titolo	Attori Coinvolti	Scopo	Relazioni
UC01	Autenticazione (RF01)	User Admin Read-only	Accedere al sistema con email e password	-
UC02	Segnala Issue (RF02)	User Admin	Creazione di una nuova issue specificando vari campi	<<include>>RF01
UC03	Visualizza lista Issue (RF03)	User Admin Read-Only	Visualizza l'elenco di issues presenti relative a un progetto specifico	<<include>>RF01
UC04	Modifica stato Issue (RF06)	User Admin	Consente di aggiornare lo stato di una issue, cambiandone i dettagli	<<include>>RF01
UC05	Esporta Issue (RF08)	User Admin Read-Only	Permette di esportare l'elenco di issues in vari formati	<<include>>RF01 <<extend>>RF03
UC06	Ricerca Issue (RF11)	User Admin Read-Only	L'utente può cercare una issue utilizzando parole chiave	<<include>>RF01 <<extend>>RF03
UC07	Archivia Issue (RF13)	Admin	Consente di achiviare le issues, rimuovendola dalla lista principale	<<include>>RF01
UC12	Gestione Utenza (RF15)	Admin	Permette di creare, modificare, eliminare altri account	<<include>>RF01

Tabella 4.2: Cockburn template Use-Case N.02, Segnalazione di una nuova issue

	A	B	C	D
1	**Use Case UC02**	Segnalazione (Aggiunta) di una nuova issue.		
2	**Goal in Context**	Si vuole segnalare una issue ad un progetto.		
3	**Preconditions**	L'utente deve essere autenticato e deve trovarsi nella vista dedicata ad un progetto. Il progetto di riferimento deve esistere nel sistema, L'utente se non e' un admin deve essere stato assegnato al progetto di riferimento.		
4	**Success End Condition**	La Issue è stata creata e salvata nel sistema con successo.		
5	**Failed End Condition**	La Issue non viene creata, viene mostrato un messaggio di errore e il sistema torna allo stato precedente.		
6	**Primary Actor**	Utente o Admin (L'utente non è read-only).		
7	**Trigger Action**	Click sul bottone "Segnala Issue".		
8	**Main Scenario**	**Step n.**	**Actor**	**System**
9		*1*	(Trigger Action) Viene premuto il bottone Segnala Issue.	
10		*2*		Viene mostrato il form per segnalare una nuova issue.
11		*3*	Vengono compilati almeno tutti i campi obbligatori	
12		*4*	Viene cliccato il bottone "Crea"	
13		*5*		Vengono verificati la validità dei dati inseriti nella form.
14		*6*		Viene salvata la issue all'interno del database del sistema.
15		*7*		(Final Action) Il form si chiude e viene lanciata una notifica di Successo.
16	**Extension #1**	*3A*	L'utente non compila tutti i campi obbligatori.	
17		*3A1*		Il sistema mostra un errore, i campi non compilati appaiono rossi.
18		*3A2*		Ritorna al passo 3.
19	**Extension #2**	*6A*		Errore interno, i dati non vengono salvati nel database del sistema.
20		*6A1*		Il form si chiude e viene lanciata una notifica di Errore
21		*6A2*		Final Action, Il caso d'uso è fallito

5

Prototipazione via Mockup

I seguenti *Mockup* rappresentano una simulazione grafica delle principali **interfacce utente** presenti nel sistema.
Hanno lo scopo di mostrare in modo chiaro il **flusso di navigazione** e le **funzionalità previste** per gli utenti. Le schermate realizzate permettono di visualizzare il **comportamento del sistema** e l'**esperienza d'uso** prima di passare all'implementazione effettiva.

5.1 Descrizione dettagliata dei Mockup

5.1.1 Schermata di Login

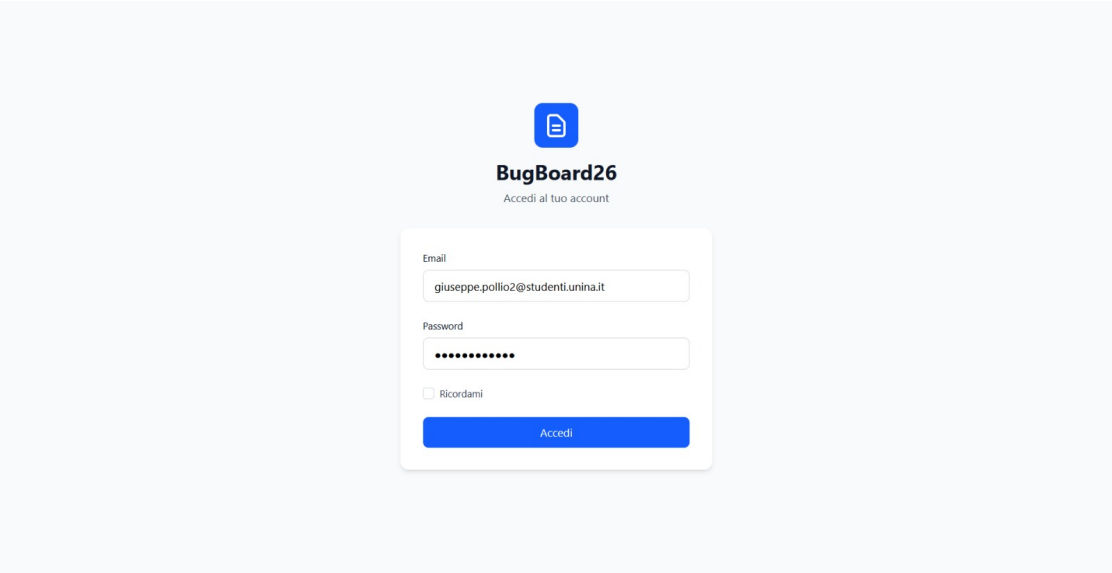

Figura 5.1: Schermata di login

La schermata raffigurata rappresenta la pagina di login al sistema.
L'utente da qui inserisce le proprie credenziali per accedere al sistema, scegliendo inoltre se mantenere la sessione attiva tramite la spunta "Ricordami"
Una volta autenticato, viene reindirizzato all'*Area progetti*

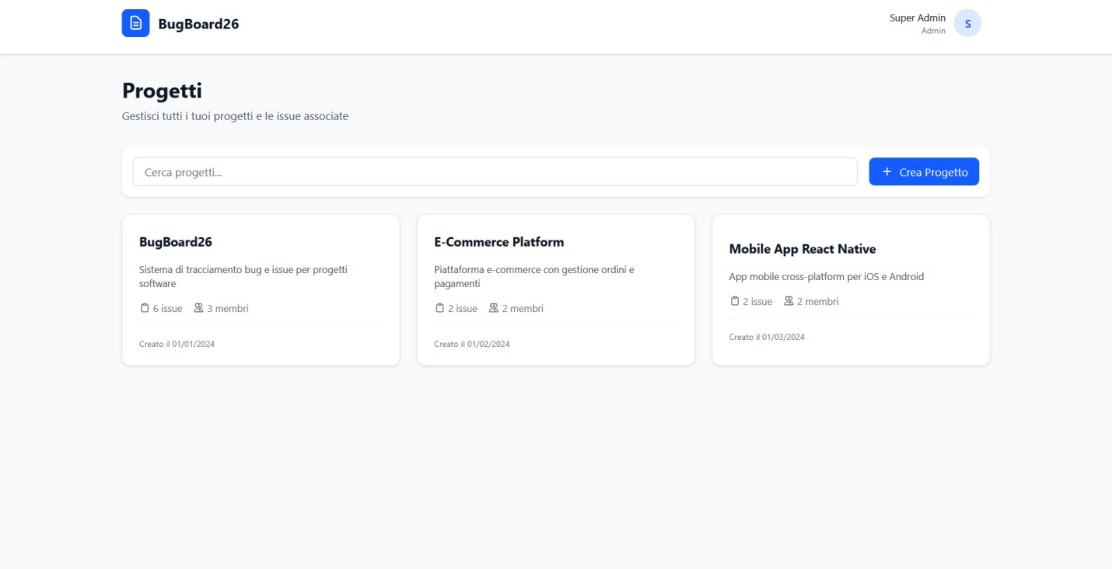

Figura 5.2: Area progetti

5.1.2 Area progetti

Una volta effettuato il login, l'utente accede all'area dei progetti, dove può visualizzare i progetti a cui partecipa.
In alto è presente una barra di ricerca per cercare i progetti e un pulsante per crearne di nuovi (visibile solo con account Admin)
Ogni progetto è caratterizzato da:

- Nome del progetto
- Breve descrizione
- Numero di *Issue* aperte
- Numero di membri del team
- Data di creazione

5.1.3 Pagina issues del Progetto

Una volta selezionato un progetto, l'utente accede alla pagina delle issues associate.
in questa pagina sono visibili tutte le segnalazioni relative al progetto selezionato, con titolo, descrizione, tipo, priorità, stato e autore.
Sopra la lista l'utente può filtrare le issues, cercarle nell'apposita barra di ricerca o ordinarle in base alla data di creazione. Inoltre è presente una spunta per visualizzare le issues archiviate.
Inoltre è presente il tasto "Crea Issue", che apre il modulo di creazione.

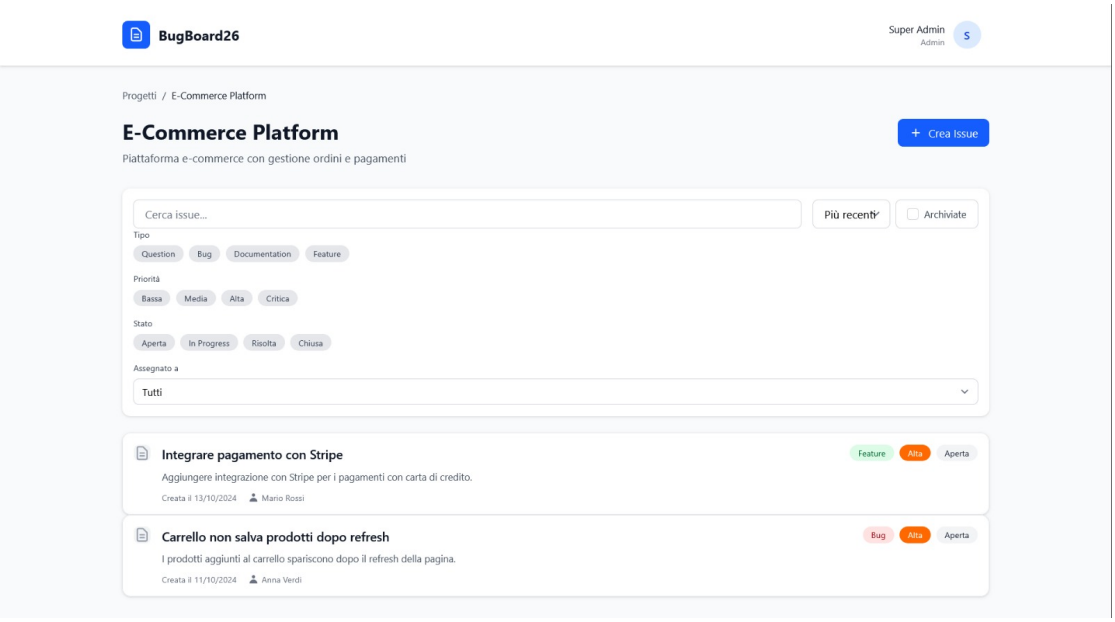

Figura 5.3: Pagina Issue
s

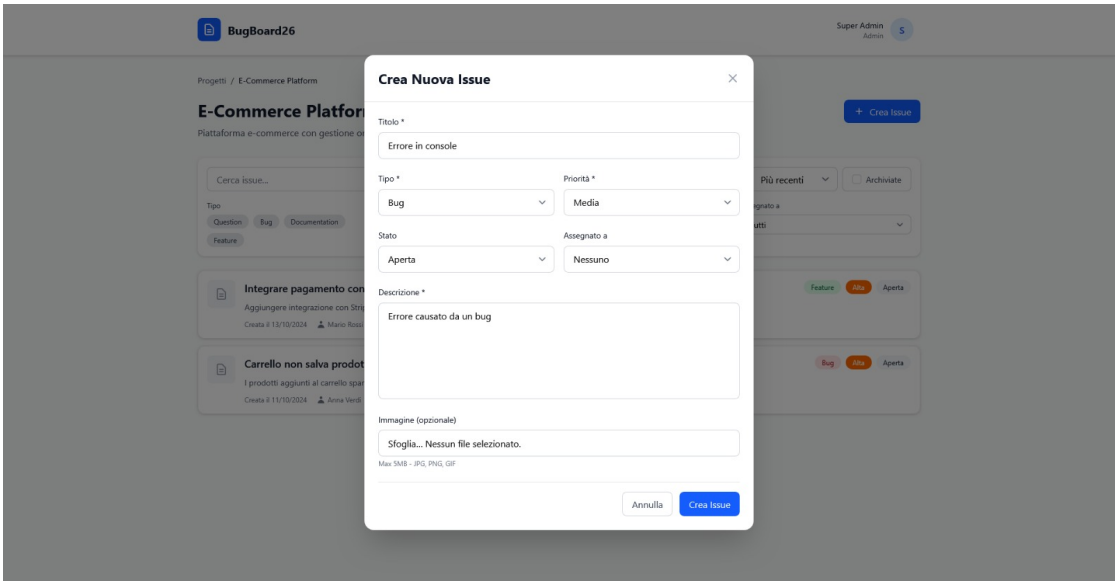

Figura 5.4: Creazione Issue

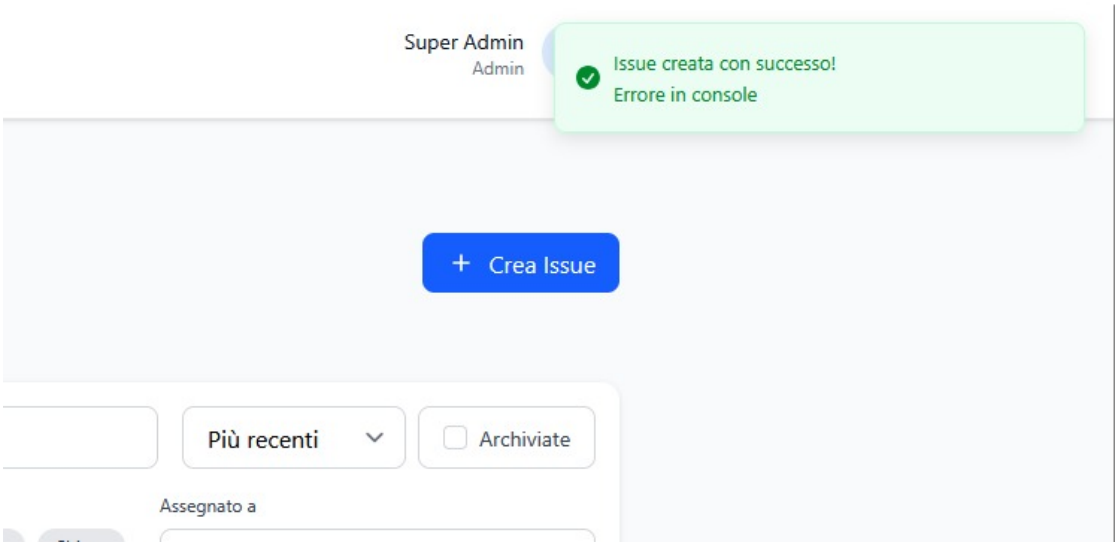

Figura 5.5: Aggiornamento lista

5.1.4 Modulo creazione Issue

Il *mockup* sopra mostra il form di inserimento di una nuova *issue*.
L'utente deve compilare alcuni campi obbligatori (titolo, tipo, priorità, descrizione), ma può anche aggiungerne di facoltativi (immagine, stato, assegnatario).
In fondo al modulo, sono presenti i pulsanti "Annulla" e "Crea Issue", che permettono rispettivamente di chiudere la schermata o di confermare la segnalazione.

5.1.5 Aggiornamento lista Issue

Il *mockup* raffigura la schermata aggiornata dopo la creazione di una nuova *issue*. La lista *issues* viene aggiornata automaticamente con la nuova voce in elenco.
Il sistema mostra inoltre una notifica di *feeback* in alto sullo schermo che conferma l'operazione di aggiunta (o segnala un eventuale errore se non è stato possibile aggiungere l'*issue*)

5.1.6 Errore durante la compilazione del modulo (creazione issue)

Durante la compilazione del modulo, se ci si dimentica di inserire tutti i campi obbligatori, si riscontra il seguente errore. Vengono mostrate i vari messaggi di errore e il sistema aspetta che l'utente corregga l'inserimento dei campi per continuare.

5.1.7 Errore interno, per qualche motivo i dati non vengono salvati nel database

Una volta finita la compilazione del modulo, dopo aver premuto il tasto "Crea", se il sistema cade in un qualsiasi errore interno allora viene lanciato il seguente messaggio di errore nella forma di una notifica.

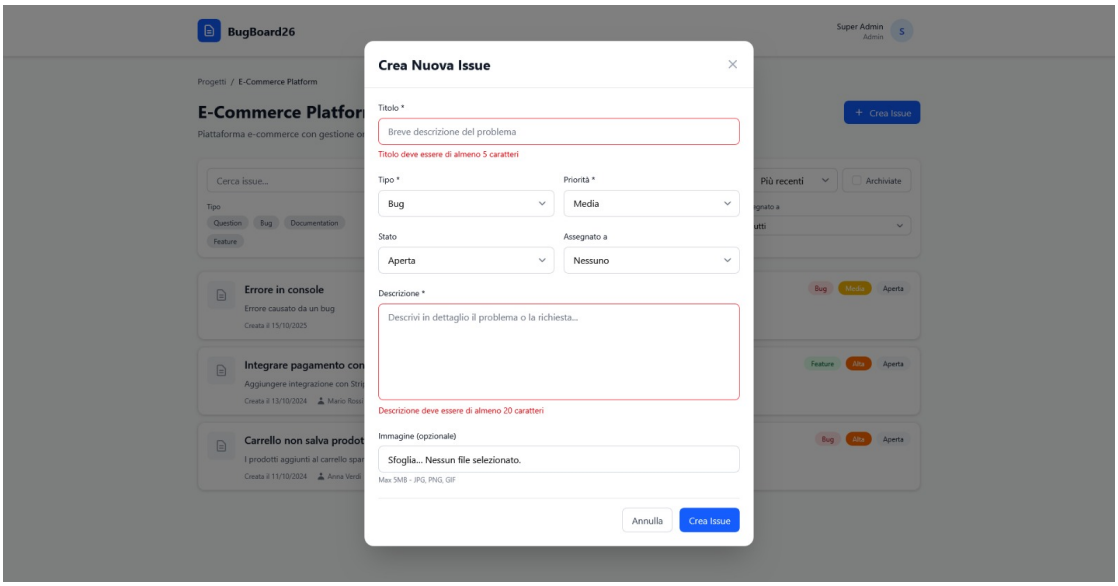

Figura 5.6: Errore 1

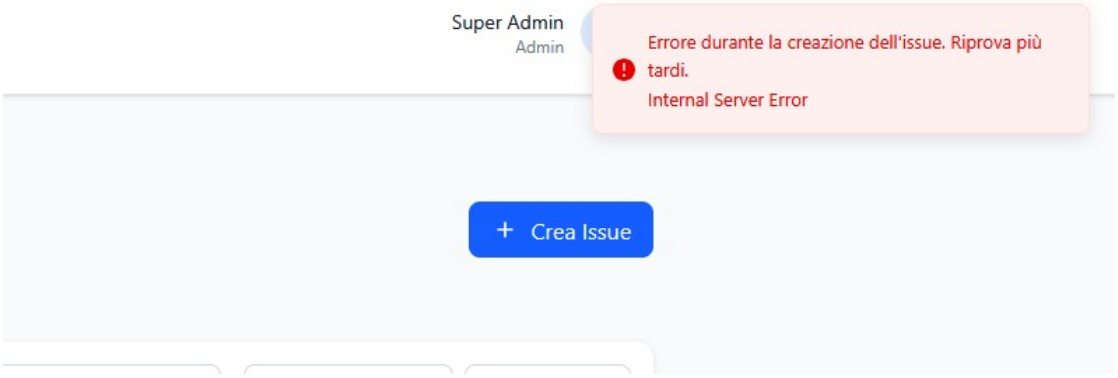

Figura 5.7: Errore 2

6

Design Del Sistema

6.1 Descrizione dell'architettura

BugBoard26 adotta un'architettura software a tre livelli, separando Frontend, Backend e Database, pensata per distinguere la logica di presentazione da quella dell'accesso dei dati.

Livello	Descrizione
Presentazione (Frontend)	Applicazione lato client,responsabile dell'interfaccia e dell'interazione con l'utente
Business Logic (Backend)	Applicazione monolitica RESTful incaricata dell'elaborazione delle richieste, della sicurezza, dell'autorizzazione e delle regole di dominio
Persistenza (Database)	Sistema relazionale utilizzato per la memorizzazione definitiva e strutturata dello stato delle entità

6.1.1 Deployment diagram

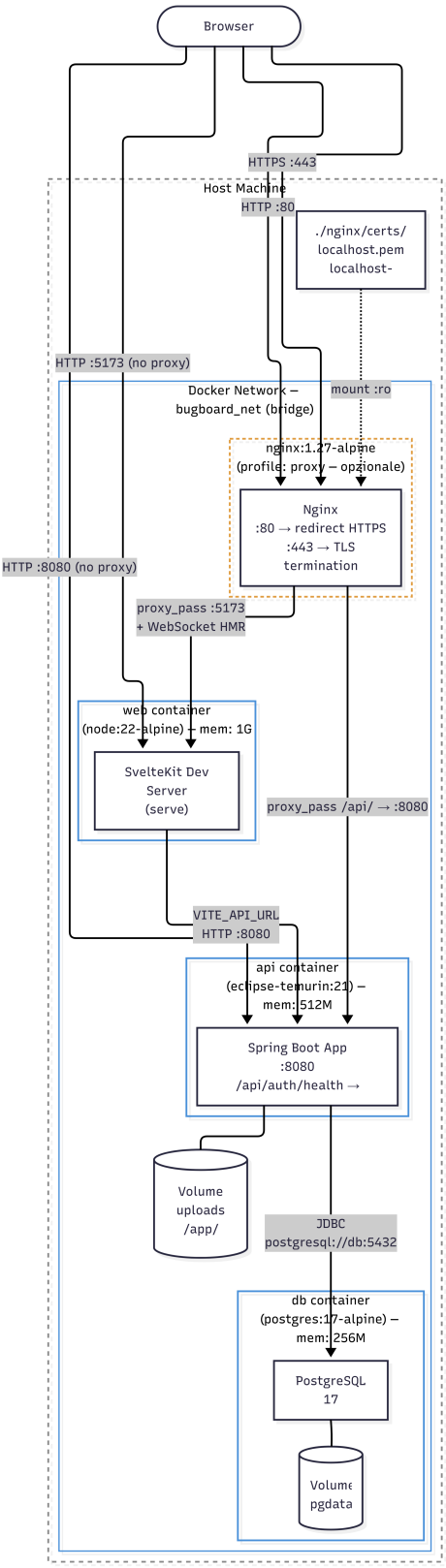

Figura 6.1: Deployment Diagram

6.2 Criteri di design adottati

- **Architettura a strati (Layered Architecture):** Le classi Spring boot sono suddivise gerarchicamente per evitare accoppiamenti forti:
  I *Controller* gestiscono le risposte HTTP, i *Service* incapsulano la logica applicativa, mentre i *Repository* effettuano manipolazioni di basso livello sul database
- **Separation of Concerns (SoC):** L'infrastruttura client-server separa l'Interfaccia Utente dalle logiche di sistema, permettendo l'aggiornamento dei due moduli in modo indipendente
- **Autuenticazione Stateless:** Il tracciamento dell'identità viene effettuato usando JSON Web Token (JWT), token crittografati generati dal client durante il login in comunicazione protetta

6.3 Scelte tecnonlogiche

6.3.1 Frontend

- **Svelte 5 e SvelteKit:** Framework usati per generare codice JavaScript efficiente e ottimizzato in fase di compilazione
- **Vite:** Tool scelto per rendere gli startup dell'ambiente di sviluppo quasi istantanei
- **Tailwind CSS:** Framework introdotto per una prototipazione scalabile, con aggiunta di plugin ufficiali per il reset form e tipografia avanzata
- **TypeScript:** usato nel client per garantire tipizzazione statica, al fine di prevenire bug in runtime

6.3.2 Backend

- **Java 23:** Versione Java utilizzata, moderna, permette l'uso di Record, utilizzati per la mappatura dei Data Transfer Object (DTO)
- **Sprint Boot 3.4.2:** Framework utilizzato per applicazioni Java, permette di autoconfigurare le dipendenze e inizializzare servlet embedded
- **Spring Data JBA:** Basato su Hibernate, usato per definire le entità e automatizzare le query dinamiche
- **Spring Security e JJWT:** Moduli responsabili di autenticazione e autorizzazione, generazione e codifica di JSON Web Tokens
- **H2 Database:** Database relazionare in memoria, utilizzato per test rapidi e isolati
- **OpenCSV e iText Core 9.1.0:** Librerie di supporto per i documenti; consentono l'esportazione di statistiche in vari formati

6.3.3 Deployment e Sviluppo

- **Docker / Docker Compose:** Gestione di frontend, backend, database e isolamento dei servizi
- **Maven:** Gestione di dipendenze e build automaton per Java

6.4 Comunicazione fra livelli

La comunicazione fra i vari livelli è basata sul protocollo HTTP/1.1, con scambio di payload in formato JSON decodificati automaticamente dal framework

- **L'autenticazione** è passata in ogni richiesta HTTP dal client tramite l'header *Authorization: Bearer <token_jwt>*
- I **flussi di esportazione** file restituiscono flussi binari o testo delimitato con i corretti header *Content-Disposition*
- Le **transazioni** che prevedono operazioni in scrittura (creazione e modifica di *Bug* o *Utenti*) sfruttano richieste mappate su precisi DTO di convalida

6.4.1 Sicurezza e Controllo degli Accessi

- **Storage sicuro di credenziali:** Le password inserite dagli Users non sono mai salvate in plaintext, bensì vengono offuscate con un *encoder crittografico* (BCrypt). Per le successive comparazioni in fase di login, si usano delle stringe *Hash*
- **Role-Based Access Control (RBAC):** Sistema basato su privilegi *GlobalRole*, per cui tentativi di esecuzione di comandi non permessi vengono intercettati dal modulo *PermissionService* e bloccati immediatamente, generando una *AccessDeniedException* per il client

7

Design del Software

7.1 Scema del Database e Modello di Dominio

Il **livello di persistenza** è gestito da Spring Data JBA, che tramite Hibernate mappa le classi Java sulle tabelle del database PostgreSQL

7.1.1 Class Diagram del database

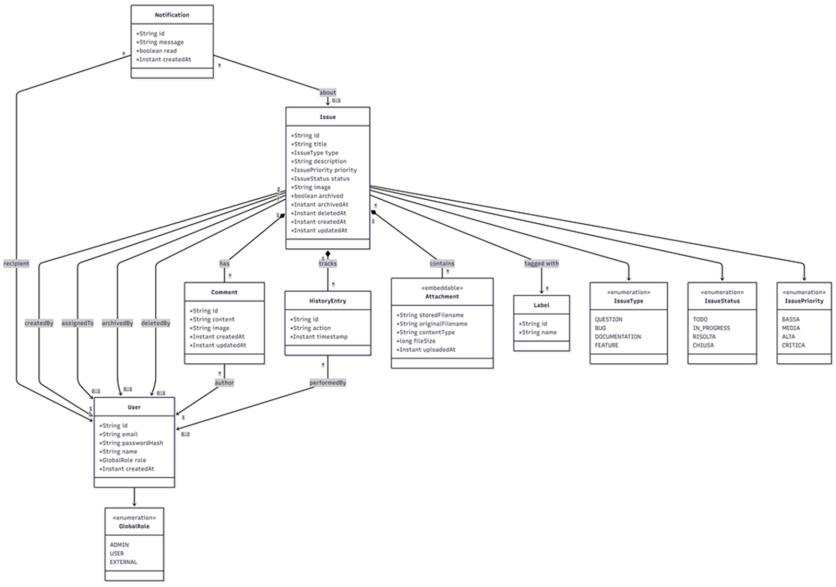

Figura 7.1: Class Diagram del database

7.1.2 Descrizione entità

- **User:** Rappresenta un utente registrato nel sistema, al suo interno contiene: email nome, hash password, la data di creazione e il *GlobalRole* associato
- **Issue:** Si tratta della task, bug, o funzionalità tracciata dal sistema. contiene data come: titolo, descrizione testuale, tipo, priorità, stato e informazioni temporali
- **Comment:** Una risposta testuale che può includere un'immagine, gestisce la conversazione di un singolo problema

- **Label:** Gestisce le etichette che possono essere assegnate ai task. Possiede un nome univoco nel database
- **HistoryEntity:** Ha lo scopo di storico modifiche, memorizzando le azioni effettuate dagli User e il timestamp in cui si è verificata
- **Notification:** Rappresenta un messaggio di avviso da parte del sistema rivolto a un User. Contiene il messaggio di notifica, data di emissione e una flag di lettura

7.1.3 Descrizione Relazioni

- **Issue - User (Reporter):** associazione * **a 1** che identifica l'User che ha creato la segnalazione
- **Issue - User (Assegnatario):** associazione * **a 1** che identifica l'User a cui è stata affidata la risoluzione di una segnalazione
- **Issue – User (Archiviatore):** associazione * **a 1** che indica l'User (amministratore) che ha archiviato la segnalazione
- **Issue - Label (Etichetta):** assosciazione * **a** * che permette di categorizzare una Issue a più Label, e di applicare lo stesso Label a più Issues diverse
- **Comment - User (Autore):** associazione * **a 1** che collega un commento all'User che l'ha creato
- **Comment - Issue (Riferimento):** assosciazione * **a 1** che collega 1 o più commenti alle Issue di appartenenza
- **Notification - User (Destinatario):** associazione * **a 1** che associa l'avviso di sistema all'User che deve riceverlo
- **Notification - Issue (Sorgente):** associazione * **a 1** opazionale, quale evento ha generato la segnalazione
- **HistoryEntry - Issue (Tracciamento):** associazione * **a 1** che collega vari storici all'Issue che ha subito variazioni nel corso del tempo
- **HystoryEntry - User (Esecutore):** associazione * **a 1** che indica l'User responsabile della modifica registrata nello storico

7.2 Organizzazione Architetturale dei Package

7.2.1 Package Diagram

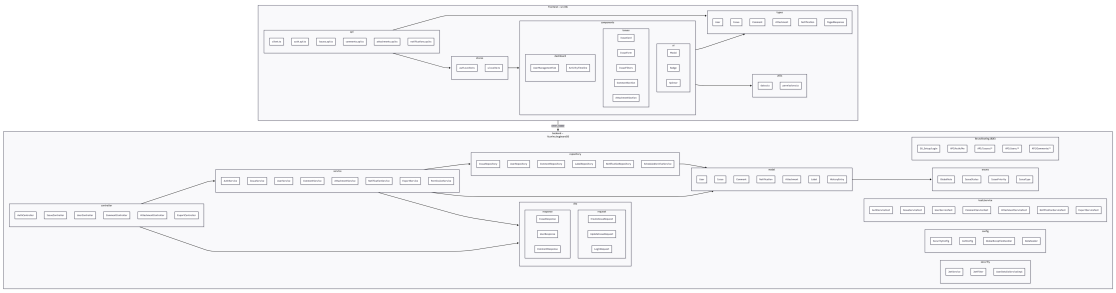

Figura 7.2: Package diagram

Il progetto adotta una suddivisione in package (**Layered Architecture**) per separare le responsabilità e facilitarne la manutenzione L'architettura del sistema è suddivisa in due aree principali che comunicano fra loro: **Backend(api)** e **Frontend(web)**

7.2.2 Backend (it.unina.bugboard26)

Racchiude tutta la logica applicativa lato server. è strutturato seguendo la logica architetturale a strati:

- `controller`: Rappresenta lo stato di presentazione delle api; riceve HTTP request dal client, delegando poi l'elaborazione allo s trato successivo
- `service`: Esegue operazioni complesse, verifica permessi e gestisce i dati prima di passasrli al database o controller
- `repository`: Gestisce lo stato di persistenza: comprende i metodi per interrogare e manipolare i dati del database, usando Spring Data JPA per astrarre le query
- `model`: contiene le entità del dominio, classi che mappano esternamente le tabelle del database
- `dto(Data Transfer Object)`: Raccoglie gli oggetti utilizzati per trasferire i dati tra client e server, nascondendo la struttura interna del database
- `security`: Si occupa della logica di autenticazione e autorizzazione, gestendo i filtri per i token JWT e l'integrità degli accessi
- `config`: Si occupa di raggruppare le configurazioni globali del sistema, quali eccezioni o configurazioni di sicurezza
- `enums`: Raccoglie i tipi enumerati condivisi a tutto il backend

7.2.3 Frontend (web)

Il package descritto riguarda l'organizzazione del codice del client web

- `routes`: Gestisce il sistema di navigazione, associando gli URL alle pagine visibili dall'utente
- `lib/components`: Raggruppa gli elementi visivi rutilizzabili dall'UI, quali bottoni, modali, schede dei task e barre di navigazione.
- `lib/api`: Si occupa di comunicare con l'esterno. Contiene i client REST che effettuano le chiamate HTTP verso il backend
- `lib/stores`: Gestisce lo stato globale dell'applicazione dal lato client
- `lib/utils`: Fornisce funzioni di supporto, come la formattazione di date o permessi lato client

7.2.4 Il flusso delle Dipendenze

Il diagramma contiene regole di dipendenza usate per evitare accoppiamenti errati:

- **Tra Macro-Package:** il pacchetto `lib/api` situato nel Frontend dipende interamente dai `controller` nel backend, comunicando tramite protocollo HTTP
- **Nel Backend:** La comunicazione è solo top-down. `controller` dipende da `service` e dai `dto`. il `service` dipende dal `repository` e `model`, mentre il `repository` unicamente dal `model`.

7.2.5 Design Pattern applicati

1. **Model-View-Controller (MVC):** Usato per API RESTful. Il *model* è rappresentato dalle entità JPA, il *Controller* intercetta le richieste di rete, la *View* è la struttura JSON rimandata al frontend
2. **DTO (Data Transfer Object):** Applicato per isolare lo schema del database dai dati scambiati in rete
3. **Repository Pattern:** Passa a Hibernate la traduzione delle query SQL
4. **Dependency Injection:** Presente nativamenete dal framework Spring, permette ai componenti (es. Service) di dichiarare le dependencies (es. Repository) usando il costruttore, che poi verranno iniettate a runtime dal framework

7.3 Sequence Diagrams

I **Sequence Diagrams** modellano il comportamento del sistema, illustrando l'ordine delle interazioni tra attori esterni e componenti dell'architettura durante l'esecuzione del programma

7.3.1 Sequence Diagram: User Login

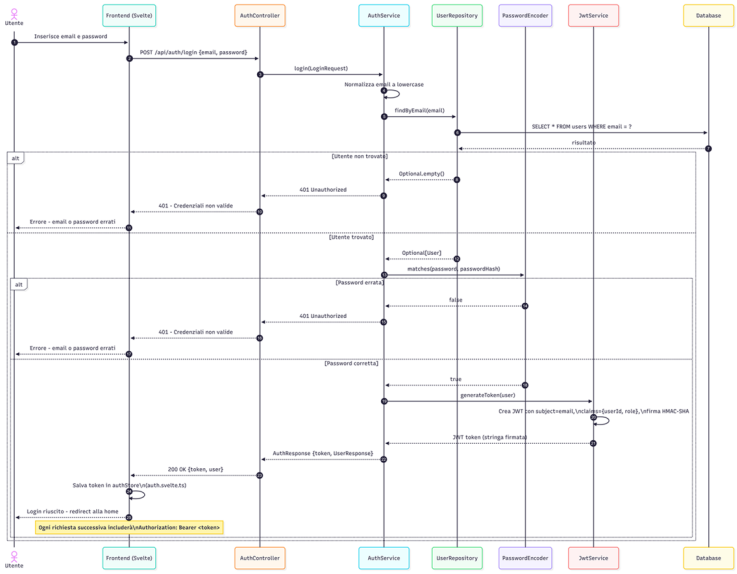

Figura 7.3: Sequence diagram Autenticazione Utente

Questo digramma descrive il flusso di verifica d'identità di un operatore, e in seguito la generazione del token crittografico per le sessioni *stateless*

Attori e componenti:

- **User:** L'essere umano che interagisce col sistema
- **User Interface:** frontend svilupptato in SvelteKit
- **AuthController:** Il punto di ingresso delle API esposte da Spring Boot
- **AuthService/JWTService:** I moduli contenenti la logica di validazione e crittografia
- **Database:** Il modulo di persistenza dove vengono conservati i dati (PostgreSQL)

Flusso degli eventi:

1. User inserisce le proprie credenziali nel login module dell'UI
2. L'applicazione client crea una richiest HTTP POST indirizzata all'APi di autenticazione, incapsulando i dati nel DTO `LoginRequest`
3. L'`AuthController` riceve la richiesta, esegue una validazione della sintassie la delega al livello successivo richiamando l'`AuthService`

4. l'`AuthService` interroga il database attraverso lo `UserRepository` cercando un account che corrisponda all'email fornito
5. Il Database restituisce l'entità utente che comprende l'hash crittografato della keyword
6. L'`AuthService` verifica la corrispondenza tra password in chiaro e l'hash salvato usando l'algoritmo BCrypt
7. Se l'esito è positivom il modulo JwtService genera un JSON Web Token firmato, incluso di privilegi di GlobalRole utente
8. Il server manda un HTTP Code 200 (success) e restituisce il JWT in un ODT AuthResponse
9. L'UI salva il token nello storage locale e reindirizza lo User alla dashboard progetti

7.3.2 Sequence Diagram: Creazione Segnalazione Bug/Issue

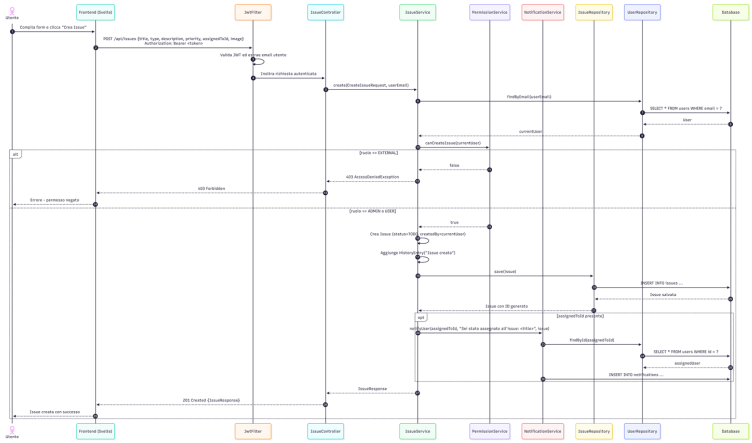

Figura 7.4: Sequence Diagram Creazione Segnalazione Bug/Issue

Questo scenario riguarda il percorso di inserimento di una Issue o Bug all'interno del sistema, analizzando il meccanismo di autorizzazione previo all'aggiunta

Attori e Componenti:

- **User Autenticato:** Operatore provvisto di JWT valido e privilegi
- **User Interface:** Componente di rendering della form
- **JwtFilter:** filtro di sicurezza apparente al framework Spring Security
- **Issue Controller:** Controller segnalazioni delle richieste per l'entità Issue
- **IssueService:** Modulo gestore della logica di business
- **Database:** Modulo di persistenza

Flusso degli Eventi:

1. L'Utente autenticato compila il modulo di segnalazione nell'UI, specificando titolo, descrizione, tipo, priorità, dunque salva
2. L'app web SvelteKit invia una richiesta HTTP POST all'API. La richiesta include: JWT nell'header di autenticazione e payload strutturato come `CreateIssueRequest`
3. La richiesta viene intercettata dal `JwtFilter`, che estrare il JWT, ne valida la firma e ricostruisce l'identità e il ruolo dell'operatore
4. Una volta superato il filter, l'`IssueController` decodifica la richiesta e la passa all'`IssueService`
5. L'`IssueService` convalida le regole di dominio (verificando che il ruolo dell'utente garantisca la creazione di Issues) e crea una nuova entità issue. impiostandone lo stato iniziale predefinito (TODO)

6. Tramite l'`IssueRepository` Il servizio comunica con il Database, richiedendone la persistenza delle entità
7. Il Database esegue l'operazione `Insert` e restituisce l'identità compresa di ID univoo generato e il timestamp
8. L'`IssueController` mappa l'entità salvata in un response DTO `IssueResponse` e lo invia al client con un codice HTTP 201 (Creato con successo)
9. L'UI riceve la conferma, mostra un toast di creazione avvenuta con successo e aggiorna la lista delle segnalazioni

8

Infrastruttura, DevOps e Continuous Integration

8.1 Contrainerizzazione tramite Docker

L'intero ecosistema BugBoard26 è containerizzato tramite **Docker** allo scopo di garantire la riproducibilità degli ambienti e risolvere il problema delle dipendenze locali
All'interno dei singoli applicativi troviamo file descrittori specifici:

- **Backend(API):** Qui un `Dockerfile` dedicato preleva il codice sorgente Java, esegure la build tramite Maven e espone il file eseguibile *.jar* su porta 8080 dentro un'immagine basata su Java 23
- **Frontend(Web):** Anche qui un `Dockerfile` gestisce le dipendenze Node.js definite in `package.json`, esegue la build del progetto SvelteKit usando *Vite* e avviia il web server su porta 5173

La gestione multi-container, necessara alla comunicazione interna, è garantita da **Docker Compose**. Il file `docker-compose.dev.yml` definisce una rete virtuale isolata e inizializza tre servizi: il database `postgres` (associato a un volute persistente per salvaguardare i dati), il server `api` e il client `web`

8.2 Automazione del Ciclo di Vita e Makefile

Per standardizzare l'esecuzione dei comandi di build, test e edeplyment il progetto comprende tutte le istruzioni operative all'interno di un `Makefile`. Ciò garantisce ai membri del progetto un'interfaccia a riga di comando semplificata per l'avvio dell'architettura locale

8.3 Pipeline di Continuous Integration (CI)

Il repository adotta pratiche di Continuous Integration per favorire lo sviluppo operatvo. Ogni volta che una commit viene spinta sul repository o viene aperta una pull request, una pipeline automatizzata valida l'integrità della codebase. I passaggli della CI includono:

1. **Check del codice** e inzializzazione degli ambienti di esecuzione come Java e Node
2. **Build Backend:** Compilazione del codice Spring Boot e risolzuone delle dependencies Maven

3. **Build Frontend:** Controllo dei TypeScript `svelte-check` e transpiling degli asset SvelteKit
4. **Quality Gate:** Interruzione della pipeline se la compilazione fallisce o una percentuale di test non viene superata

9

Piano di collaudo

9.1 Metodologia e Perimetro di Collaudo

La stategia di Quality Assurance si concentra esclusivamente su Backend (server. L'aprroccio è quello dell'**Unit Testing** tradizionale, supportato ulteriormente dal framework **JUnit 5**, con l'obiettivo di validare i componenti centrali dell'applicazione simulando l'interazione fra moduli e isolando le dipendenze esterne tramite l'utilizzo di Mock.
Per garentire una copertura qualitativa, i testi sono incentrati su metodi ad **alta complessità logica**, non banali, scartando getter, setter e metodi delegati. Sono stati individuati 3 metodi essenziali, caratterizzati da regole di dominio rigorose e **firme con 2 o più parametri** in ingresso

9.2 Strategie di Isolamento e Classi di Equivalenza (Black Box)

Per metodi compessi che posseggono effetti collaterali o che non ritornano un valore esplicito, si è fatto uso della libreria **Mockito** per assicurare rapidità nell'escuzione e assenza di side-effect.
Tutti gli oggetti di tipo `Repository` responsabili dell' accesso dati fisici sono stati rimpiazzati con sosia Mock simulati tramite annotazioni `@Mock` e `@InjectMocks`, per fare in modo di concentrare la valutazione esclusivamente sull'algoritmo Java.

- **Esempio applicato(`IssueService.update`)**: L'aggiornamento di un ticket aziona funzionalità di notifiche asincrone. La pianificazione ha dunque previsto test case specifici per verificare la cardinalità delle chiamate esterne, usando l'asserzione `verify()` di Mockito per garantire che il `NotificationService` venga invocato una sola volta in caso di aggiornamento da parte di terzi, e `verify(...,never())` per verificare l'assenza di chiamate qualora l'aggiornamento venga effettuato dall'autore stesso della segnalazione, prevenendo spam

Per ottimizzare la copertura evitando ridondanze durante l'esecuzione, si sono selezionati gli input basandosi sulla tecnica delle **Classi di Equivalenza**. Per ogni metodo sono stati dunque testati i percorsi ideali (Happy Path), percorsi di fallimento a causa di assenza di permessi (Error Path) e la gestione dei casi limite (Boundary Conditions), assumendo che il sistema si comporti in modo analogo per tutti i valori appartenenti allo stesso raggruppamento

9.3 Metodi Sottoposti a Test e Casi Pianificati

L'attività di collaudo ha verificato il comportamento di 3 parti cruciali del sistema:

9.3.1 Servizio di Gestione Segnalazioni (`IssueService.update()`)

- **Metodo testato:**
  `update(String id, UpdateIssueRequest req, User currentUser)`
- **Logica di Collaudo:** Verifica la composizione complessa tra i componenti. Il metodo deve prima accertarsi dei permessi dell'utente, poi applicare i cambiamenti di stato, e infine, tramite il modulo `NotificationService`, iniziare le comunicazioni asincrone
- **Partizione degli Input:** Le richieste valide `UpdateIssueRequest` sono divise in due classi di equivalenza: cambiamenti effettuati da terzi (che necessitano una notifica) e cambiamenti effettuati dall'autore (che la sopprimono)

ID Caso	Descrizione dello Scenario	Risultato Atteso
TC-ISS-UPD-01	Happy Path (Terzi): Utente assegnatario invia un DTO valido per cambiare lo stato in RISOLTA.	Il repository salva l'entità. Tramite verify(mock, times(1))si fa in modo che NofiticationService venga inviato una sola volta
TC-ISS-UPD-02	Happy Path (Autore): L'autore originale della Issue modifica la descrizione o lo stato.	Il repository salva l'entità. verify(mock, nevere()) garantisce che nessuna notifica venga generata, evitando spam
TC-ISS-UPD-03	Error Path (Privilegi): Utente base tenta di archiviare la segnalazione impostando il flag archived=true.	Si interrompe l'esecuzione, viene sollevato AccessDeniedException e non vengono effettuate chiamate di salvataggio verso il DB
TC-ISS-UPD-04	Boundary Condition: L'ID fornito non corrisponde a nessuna Issue esistente nel database.	Il repository simulato restituisce "vuoto", con il sollevamento controllato di EntityNotFoundException

9.3.2 Servizio di Autorizzazione: PermissionService.canModifyIssue()

- **Firma del metodo:** `canModifyIssue(User user, Issue issue)`
- **Logica di collaudo:** Validazione della matrice dei privilegi *RBAC* (Role Based Access Control). il metodo deve prevenire la *Privilege Escalation* calcolando i diritti d'accesso basati su `GlobalRole` o sull'assegnazione specifica
- **Partizione degli Input:** il parametro `USER` è stato suddiviso in diverse classi di equivalenza: `Admin`(Ruolo assoluto), `USER` assegnato alla issue (Ruolo Relativo), `USER` non assegnato (Ruolo Ristretto) e `EXTERNAL` (Ruolo Esterno)

ID Caso	Descrizione dello Scenario	Risultato Atteso
TC-PERM-MOD-01	Authorized (Ruolo Assoluto): Valutazione di un operatore dotato di GlobalRole.ADMIN	L'asserzione assertTrue conferma che il metodo garantisce l'accesso ignorando l'assegnazione della Issue
TC-PERM-MOD-02	Authorized (Ruolo Relativo): Valutazione di un operatore USER il quale ID coincide con l'assigned_to_id della Issue	L'asserzione assertTrue conferma l'autorizzazione basata sulla paternità/ assegnazione del task
TC-PERM-MOD-03	Unauthorized (Nessuna Relazione): Valutazione di un operatore USER estraneo alla Issue in esame	L'asserzione assertFalse conferma il blocco della modifica, garantendo l'isolamento dei task tra team
TC-PERM-MOD-04	Unauthorized (Ruolo Esterno): Valutazione di un operatore con ruolo EXTERNAL (es. cliente in sola lettura)	L'asserzione assertFalse conferma la negazione assoluta dei privilegi di scrittura, indipedentemente dall'Issue passata

9.3.3 Servizio di Esportazione: ExportService.exportCsv()

- **Firma del metodo:** `exportCsv(List<Issue> issues, ExportRequest filters)`
- **Logica di Collaudo:** Elaborazione sicura di una collezione di oggetti in un array di Byte (flusso) conforme allo standard CSV, applicando eventuali filtri di raffinamento
- **Partizione degli Input:** La lista di entità di ingresso `List<Issue>` è stata testata sia con valori validi e popolati, sia con casi limite come lista vuota o entità contenenti null, per prevenire crash a runtime e valutare la stabiltà dell'algoritmo di parsing

ID Caso	Descrizione dello Scenario	Risultato Atteso
TC-EXP-CSV-01	Happy Path: Iniezione di una lista popolata con diverse Issue con tutti i metadati	Il metodo restituisce un Array di Byte non nullo. L'asserzione verifica che la stringa modificata contenga il numero di righe atteso, incluso di Header
TC-EXP-CSV-02	Boundary Condition (Lista Vuota): Iniezione di una Collections.emptyList()	Non vengono sollevate eccezioni. Il metodo elabora il caso limite restituendo un CSV valido contenente solo l'Header
TC-EXP-CSV-03	Boundary Condition (Null Safety): Iniezione di Issue contenenti campi testuali opzionali non valorizzati	L'algoritmo non incorre in NullPointerException. I campi vuoti vengono formattato nel CSV come stringhe vuote " "
TC-EXP-CSV-04	Filter Path: Applicazione di un ExportRequest per parametri di delimitazione personalizzati (es. il separatore ; al posto della ,)	L'asserzione verifica che la sintassi del documento rispetti le direttive del DTO di configurazione passato in input

10

Analisi statistica e Qualità del Codice (SonarQube)

Per affiancare il testing dinamico descritto nel capitolo precedente e garantire manutenibilità a lungo termine dell'architettura Backend, il codice sorgente viene sottoposto a ispezione statica tramite **SonarQube**.
Questo strumento è integrato nella pipeline di CI e valuta la codebase rispetto a un *Quality Gate*, fornendo delle metriche precise sul *Technical Debt* e validando le aree di logica complessa sottoposte già a Unit Testing
L'analisi produce un report qualitativo incentrato su diversi assi metrici, con attenzione a servizi essenziali dell'applicazione:

10.0.1 Security and Vulnaribilities (Sicurezza)

Al fine di garantire che non siano presenti vulnerabilità note o cattive pratiche architetturali, il codice viene scansionato. In riferimento al `PermissionService`, valutato secondo il metodo `canModifyIssue`, SonarQube verifica l'assenza di *Security Hotspost*.
Garantisce che non ci siano bypass logici o credenziali hardcoded dentro la matrice di visualizzazone dei privilegi RBAC e che l'implementazione di controlli autorizzativi non venga bypassatada tentativi di Privilege Escalation

10.0.2 Reliability and Bugs (Affidabilità)

Quest'ispezione rileva potenziali errori logici aruntime che potrebbero sfuggire anche ad alcuni test.
Un'enfatizzazione particolare è rivolta all'`ExportService` e al metodo `exportCsv`. SonarQube esamina il codice per evitare la deferenzazione di null pointer, assicurandosi che non ci siano *Resource Leaks*

10.0.3 Manintainability and Code Smells (Manutenibilità e Complessità)

L'analisi identifica porzioni di codice funzionanti ma strutturalmente carenti.
Il metodo `IssueService.update` poichè gestisce permessi, DB, variazioni di stato, notifiche potrebbe accumulare un'elevata complessità ciclomatica (troppi if/else annidati). SonarQube per monitorare quest'indicatore possiede una soglia di guardia che blocca le pull request che superano una certa complessità promuovendo il refactoring. Inoltre verifica che i DTO non presentino parametri non utilizzati

10.0.4 Test Coverage (Copertura del Codice)

Si può quantificare visivamente la percentuale esatta di linee di codice e rami condizionali effettivamente attraversati durante il testing (**Branch Coverage**) utilizzando l'integrazione tra i report generati da JaCoCo e SonarQube.
Grazie all'applicazione delle *Classi di Equivalenza* e della *Behavior Verification* sui metodi `update`, `canModifyIssue` e `exportCsv` SonarQube si assicura che gli Happy Path, gli Error Path e Boundary Conditions del layer `Service` raggiungano una più ampia copertura, in linea con gli standard qualitativi del progetto.

11

Gestione della Configurazione e Versioning

La gestione del codice sorgente, l'architettura e la collaborazione tra membri è stato coordinato tramite il sistema di versioning control **Git**.
L'hosting del repository è stato invece affidato alla piattaforma **GitHub**, che possiede l'intera codebase, includendo layer applicativi (API e Web), script di automazione *Makefile* e i *Docker Compose* descrittori dell'infrastruttura.

11.0.1 Repository Ufficiale

L'intero codice sorgente del progetto BugBoard26, compresa la cronologia delle modifiche e configuration files è visualizzabile al seguente indirizzo:

- **GitHub Repository:** `https://github.com/polliog/bugboard26`

11.0.2 Metriche, Statistiche, Workflow

GitHub traccia analiticamente l'andamento dei lavori analzziando i vari contributi dei membri. Le seguenti statistiche, estratte dalla dashboard *Insights* della repository, evidenziano la continuità dell'attività di implementazione:

- **Contributors:** Il progetto è stato sviluppato da 3 contributori, che hanno operato in parallelo su componenti diverse dell'architettura, garntendo una separazione tra task di Backend e Frontend
- **Volume dei Commit:** Dall'inizializzaione del progetto fino alla sua fine, sono state registrate 62 commits totali. Questo volume rispecchia uno sviluppo di tipo incrementale, con salvataggi atomici, coesi e facilmente tracciabili in caso di anomali
- **Gestione tramite Pull Request:** Per garantire la stabilità dell'applicativo, il codice all'interno del branch main è stato mediato attraverso l'uso di Pull Request. In questo modo è stato possibile effettuare ogni merge in seguito all'esito positivo della pipeline di Continuous Integration e al superamento del Quality Gate imposto da SonarQube
- **Strategia di Branching:** Il ciclo di vita del codice ha seguito un modello derivato dal *GitHub Flow*, dove ogni funzionalità nuova di risoluzione bug è stata sviluppata all'interno di *feature branch* isolati, per proteggere il branch `main`. Nella fase finale del progetto, questi branch sono stati sistematicamente integrati all'interno del main e successivamente rimossi.